

8. ResultSet interface

The SQL statements that read data from a database query, return the data in a result set. The SELECT statement is the standard way to select rows from a database and view them in a result set. The *java.sql.ResultSet* interface represents the result set of a database query.

Note: By default, ResultSet object can be moved forward only and it is not updatable.

A ResultSet object maintains a cursor that points to the current row in the result set. The term "result set" refers to the row and column data contained in a ResultSet object.

The methods of the ResultSet interface can be broken down into three categories –

Navigational methods – Used to move the cursor around.

Get methods – Used to view the data in the columns of the current row being pointed by the cursor.

Update methods – Used to update the data in the columns of the current row. The updates can then be updated in the underlying database as well.

The cursor is movable based on the properties of the ResultSet. These properties are designated when the corresponding Statement that generates the ResultSet is created.

JDBC provides the following connection methods to create statements with desired ResultSet
createStatement(int RSType, int RSConcurrency);
prepareStatement(String SQL, int RSType, int RSConcurrency);
prepareCall(String sql, int RSType, int RSConcurrency);

The first argument indicates the type of a ResultSet object and the second argument is one of two ResultSet constants for specifying whether a result set is read-only or updatable.

8.1 Type of ResultSet

The possible RSType are given below. If you do not specify any ResultSet type, you will automatically get one that is TYPE_FORWARD_ONLY.

Type	Description
ResultSet.TYPE_FORWARD_ONLY	The cursor can only move forward in the result set.
ResultSet.TYPE_SCROLL_INSENSITIVE	The cursor can scroll forward and backward, and the result set is not sensitive to changes made by others to the database that occur after the result set was created.

ResultSet.TYPE_SCROLL_SENSITIVE.

The cursor can scroll forward and backward, and the result set is sensitive to changes made by others to the database that occur after the result set was created.

8.2 Concurrency of ResultSet

The possible RSConcurrency are given below. If you do not specify any Concurrency type, you will automatically get one that is CONCUR_READ_ONLY.

Concurrency	Description
ResultSet.CONCUR_READ_ONLY	Creates a read-only result set. This is the default
ResultSet.CONCUR_UPDATABLE	Creates an updateable result set.

All our examples written so far can be written as follows, which initializes a Statement object to create a forward-only, read only ResultSet object –

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class DafaaultResultSetExample {
    public static void main(String[] args) {
        try {
            // Load and Register the Driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // We can also use url like jdbc:mysql:///organization
            // If MySQL is running in localhost and default port 3306
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/organization",
                "root", "root");

            //Statement stmt = con.createStatement();
            Statement stmt = con.createStatement(
                ResultSet.TYPE_FORWARD_ONLY, ResultSet.CONCUR_READ_ONLY);
            ResultSet rs = stmt.executeQuery("SELECT * FROM employee");

            while (rs.next()) {
                System.out.println(rs.getInt(1) + "\t" + rs.getString(2)
                    + "\t" + rs.getString(3) + "\t" + rs.getInt(4));
            }

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Output:

```

1   John Doe   Software Engineer   60000
2   Jane Smith Senior Developer   75000
3   Robert Brown Project Manager   90000
4   Emily White QA Engineer   55000
5   Michael Green DevOps Engineer   65000

```

8.3 Navigating a Result Set

There are several methods in the ResultSet interface that involve moving the cursor, including

S.N. Methods & Description

- 1 **public void beforeFirst() throws SQLException**
Moves the cursor just before the first row.
- 2 **public void afterLast() throws SQLException**
Moves the cursor just after the last row.
- 3 **public boolean first() throws SQLException**
Moves the cursor to the first row.
- 4 **public void last() throws SQLException**
Moves the cursor to the last row.
- 5 **public boolean absolute(int row) throws SQLException**
Moves the cursor to the specified row.
- 6 **public boolean relative(int row) throws SQLException**
Moves the cursor the given number of rows forward or backward, from where it is currently pointing.
- 7 **public boolean previous() throws SQLException**
Moves the cursor to the previous row. This method returns false if the previous row is off the result set.
- 8 **public boolean next() throws SQLException**
Moves the cursor to the next row. This method returns false if there are no more rows in the result set.
- 9 **public int getRow() throws SQLException**
Returns the row number that the cursor is pointing to.
- 10 **public void moveToInsertRow() throws SQLException**
Moves the cursor to a special row in the result set that can be used to insert a new row into the database. The current cursor location is remembered.
- 11 **public void moveToCurrentRow() throws SQLException**

Moves the cursor back to the current row if the cursor is currently at the insert row; otherwise, this method does nothing

Navigating to first, last and next Record in a ResultSet Example

In this example, we've four static strings containing a database connection url, username, password and a SELECT query. Now using DriverManager.getConnection() method, we've prepared a database connection. Once connection is prepared, we've created a Statement object using connection.createStatement() method. While creating Statement Object, we've used ResultSet types as TYPE_SCROLL_INSENSITIVE and CONCUR_READ_ONLY, then using statement.executeQuery(), the SELECT Query is executed and result is stored in a resultset.

In first step, we've moved resultset cursor to the last row using ResultSet.last() method and printed the last record. In second step, we've moved cursor to the first row using ResultSet.first() method and printed the first record. As last step, we moved cursor to next record using ResultSet.next() method and printed the record.

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class NavigatingResultSetExample1 {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/organization", "root", "root");

            Statement stmt = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
                ResultSet.CONCUR_READ_ONLY);
            ResultSet rs = stmt.executeQuery("SELECT * FROM employee");

            // Move cursor to the last row.
            System.out.println("Moving cursor to the last...");
            rs.last();

            // Extract data from result set
            System.out.println("Displaying record...");
            System.out.println("Id: " + rs.getInt("id") + ", Name: " +
                rs.getString("name") + ", Designation: " + rs.getString("designation")
                + ", Salary: " + rs.getInt("salary"));

            // Move cursor to the first row.
            System.out.println("Moving cursor to the first row...");
            rs.first();

            // Extract data from result set
            System.out.println("Displaying record...");
            System.out.println("Id: " + rs.getInt("id") + ", Name: " +
                rs.getString("name") + ", Designation: " + rs.getString("designation")
                + ", Salary: " + rs.getInt("salary"));

            System.out.println("Moving cursor to the next row...");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```

        rs.next();

        // Extract data from result set
        System.out.println("Displaying record...");
        System.out.println("Id: " + rs.getInt("id") + ", Name: " +
            rs.getString("name") + ", Designation: " + rs.getString("designation")
                + ", Salary: " + rs.getInt("salary"));

        con.close();

    } catch (Exception e) {
        System.out.println(e);
    }
}

```

Output:

```

Moving cursor to the last...
Displaying record...
Id: 5, Name: Michael Green, Designation: DevOps Engineer, Salary: 65000
Moving cursor to the first row...
Displaying record...
Id: 1, Name: John Doe, Designation: Software Engineer, Salary: 60000
Moving cursor to the next row...
Displaying record...
Id: 2, Name: Jane Smith, Designation: Senior Developer, Salary: 75000

```

Navigating to a particular Record in a ResultSet Example

In this example, we've four static strings containing a database connection url, username, password and a SELECT query. Now using DriverManager.getConnection() method, we've prepared a database connection. Once connection is prepared, we've created a Statement object using connection.createStatement() method. While creating Statement Object, we've used ResultSet types as TYPE_SCROLL_INSENSITIVE and CONCUR_READ_ONLY, then using statement.executeQuery(), the SELECT Query is executed and result is stored in a resultset.

In first step, we've moved resultset cursor to a particular row using ResultSet.absolute() method and printed the record. In second step, we've moved cursor to the first row using ResultSet.first() method and printed the first record. As last step, we moved cursor to next record using ResultSet.next() method and printed the record.

```

package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class NavigatingResultSetExample2 {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/organization", "root", "root");

```

```

Statement stmt = con.createStatement(
    ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
ResultSet rs = stmt.executeQuery("SELECT * FROM employee");

// Move cursor to the third row.
System.out.println("Moving cursor to the 3rd row...");
rs.absolute(3);

// Extract data from result set
System.out.println("Displaying record of 3rd row...");
System.out.println("Id: " + rs.getInt("id") + ", Name: " +
    rs.getString("name") + ", Designation: " +
    rs.getString("designation") + ", Salary: " +
    rs.getInt("salary"));

// Move cursor to the first row.
System.out.println("Moving cursor to the first row...");
rs.first();

// Extract data from result set
System.out.println("Displaying record...");
System.out.println("Id: " + rs.getInt("id") + ", Name: " +
    rs.getString("name") + ", Designation: " +
    rs.getString("designation") + ", Salary: " +
    rs.getInt("salary"));

System.out.println("Moving cursor to the next row...");
rs.next();

// Extract data from result set
System.out.println("Displaying record...");
System.out.println("Id: " + rs.getInt("id") + ", Name: " +
    rs.getString("name") + ", Designation: " +
    rs.getString("designation") + ", Salary: " +
    rs.getInt("salary"));

    con.close();
} catch (Exception e) {
    System.out.println(e);
}
}
}

```

Output:

```

Moving cursor to the 3rd row...
Displaying record of 3rd row...
Id: 3, Name: Robert Brown, Designation: Project Manager, Salary: 90000
Moving cursor to the first row...
Displaying record...
Id: 1, Name: John Doe, Designation: Software Engineer, Salary: 60000
Moving cursor to the next row...
Displaying record...
Id: 2, Name: Jane Smith, Designation: Senior Developer, Salary: 75000

```

8.4 Viewing a Result Set

The ResultSet interface contains dozens of methods for getting the data of the current row.

There is a get method for each of the possible data types, and each get method has two versions

- One that takes in a column name.
- One that takes in a column index.

For example, if the column you are interested in viewing contains an int, you need to use one of the getInt() methods of ResultSet –

S.N. Methods & Description

- | | |
|---|--|
| 1 | public int getInt(String columnName) throws SQLException
Returns the int in the current row in the column named columnName. |
| 2 | public int getInt(int columnIndex) throws SQLException
Returns the int in the current row in the specified column index. The column index starts at 1, meaning the first column of a row is 1, the second column of a row is 2, and so on. |

Similarly, there are get methods in the ResultSet interface for each of the eight Java primitive types, as well as common types such as java.lang.String, java.lang.Object, and java.net.URL.

There are also methods for getting SQL data types java.sql.Date, java.sql.Time, java.sql.Timestamp, java.sql.Clob, and java.sql.Blob. Check the documentation for more information about using these SQL data types.

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class ViewingResultSetExample {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/organization", "root", "root");

            Statement stmt = con.createStatement(
                ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
            ResultSet rs = stmt.executeQuery("SELECT * FROM employee");

            System.out.println("Viewing Records using column names");
            while (rs.next()) {
                System.out.println(rs.getInt("id") + "\t" +
                    rs.getString("name") + "\t" + rs.getString("designation") +
                    "\t" + rs.getInt("salary"));
            }

            rs.beforeFirst(); // Moving cursor to initial stage
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```

        System.out.println("\nViewing Records using column index");
        while (rs.next()) {
            System.out.println(rs.getInt(1) + "\t" + rs.getString(2) +
                               "\t" + rs.getString(3) + "\t" + rs.getInt(4));
        }

        con.close();
    } catch (Exception e) {
        System.out.println(e);
    }
}

```

Output:

```

Viewing Records using column names
1      John Doe      Software Engineer    60000
2      Jane Smith    Senior Developer     75000
3      Robert Brown  Project Manager     90000
4      Emily White   QA Engineer         55000
5      Michael Green DevOps Engineer     65000

Viewing Records using column index
1      John Doe      Software Engineer    60000
2      Jane Smith    Senior Developer     75000
3      Robert Brown  Project Manager     90000
4      Emily White   QA Engineer         55000
5      Michael Green DevOps Engineer     65000

```

8.5 Updating a Result Set

The ResultSet interface contains a collection of update methods for updating the data of a result set. As with the get methods, there are two update methods for each data type –

- One that takes in a column name.
- One that takes in a column index.

For example, to update a String column of the current row of a result set, you would use one of the following updateString() methods –

S.N. Methods & Description

- 1 **public void updateString(int columnIndex, String s) throws SQLException**
Changes the String in the specified column to the value of s.
- 2 **public void updateString(String columnName, String s) throws SQLException**
Similar to the previous method, except that the column is specified by its name instead of its index.

There are update methods for the eight primitive data types, as well as String, Object, URL, and the SQL data types in the java.sql package.

Updating a row in the result set changes the columns of the current row in the ResultSet object, but not in the underlying database. To update your changes to the row in the database, you need to invoke one of the following methods.

S.N. Methods & Description

- 1 **public void updateRow()**
Updates the current row by updating the corresponding row in the database.
- 2 **public void deleteRow()**
Deletes the current row from the database
- 3 **public void refreshRow()**
Refreshes the data in the result set to reflect any recent changes in the database.
- 4 **public void cancelRowUpdates()**
Cancels any updates made on the current row.
- 5 **public void insertRow()**
Inserts a row into the database. This method can only be invoked when the cursor is pointing to the insert row.

Updating and Inserting Records using ResultSet Example

In this example, we've four static strings containing a database connection url, username, password and a SELECT query. Now using DriverManager.getConnection() method, we've prepared a database connection. Once connection is prepared, we've created a Statement object using connection.createStatement() method. While creating Statement Object, we've used ResultSet types as TYPE_SCROLL_INSENSITIVE and CONCUR_UPDATABLE, then using statement.executeQuery(), the SELECT Query is executed and result is stored in a resultset.

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class UpdatingResultSetExample {

    public static void printResultSet(ResultSet rs) throws SQLException {
        // Ensure we start with first row
        rs.beforeFirst();
        while (rs.next()) {
            System.out.println("Id: " + rs.getInt("id") + ", Name: " +
                               rs.getString("name") + ", Designation: " +
                               rs.getString("designation") + ", Salary: " +
                               rs.getInt("salary"));
        }
        System.out.println();
    }
}
```

```

public static void main(String[] args) {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");

        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/organization", "root", "root");

        Statement stmt = con.createStatement(
            ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_UPDATABLE);
        ResultSet rs = stmt.executeQuery("SELECT * FROM employee");

        System.out.println("List result set for reference....");
        printResultSet(rs);

        // Loop through result set and add 1000 in salary
        // Move to Before first position so while-loop works properly

        rs.beforeFirst();
        // Extract data from result set
        while (rs.next()) {
            // Retrieve by column name
            int newSalary = rs.getInt("salary") + 1000;
            rs.updateInt("salary", newSalary);
            rs.updateRow();
        }

        System.out.println("List result set showing new salary...");
        printResultSet(rs);

        // Insert a record into the table.
        System.out.println("Inserting a new record...");

        // Move to insert row and add column data with updateXXX()
        rs.moveToInsertRow();
        rs.updateInt("id", 6);
        rs.updateString("name", "Rocky");
        rs.updateString("designation", "Team Lead");
        rs.updateInt("salary", 95000);

        // Commit row
        rs.insertRow();

        System.out.println("List result set showing new set...");
        printResultSet(rs);

        // Deleting the added row.
        rs.last();
        rs.deleteRow();

        System.out.println("List result set showing row after deletion.");
        printResultSet(rs);

        con.close();
    } catch (Exception e) {
        System.out.println(e);
    }
}

```

Output:

```

List result set for reference....
Id: 1, Name: John Doe, Designation: Software Engineer, Salary: 60000
Id: 2, Name: Jane Smith, Designation: Senior Developer, Salary: 75000

```

```
Id: 3, Name: Robert Brown, Designation: Project Manager, Salary: 90000
Id: 4, Name: Emily White, Designation: QA Engineer, Salary: 55000
Id: 5, Name: Michael Green, Designation: DevOps Engineer, Salary: 65000
```

List result set showing new salary...

```
Id: 1, Name: John Doe, Designation: Software Engineer, Salary: 61000
Id: 2, Name: Jane Smith, Designation: Senior Developer, Salary: 76000
Id: 3, Name: Robert Brown, Designation: Project Manager, Salary: 91000
Id: 4, Name: Emily White, Designation: QA Engineer, Salary: 56000
Id: 5, Name: Michael Green, Designation: DevOps Engineer, Salary: 66000
```

Inserting a new record...

List result set showing new set...

```
Id: 1, Name: John Doe, Designation: Software Engineer, Salary: 61000
Id: 2, Name: Jane Smith, Designation: Senior Developer, Salary: 76000
Id: 3, Name: Robert Brown, Designation: Project Manager, Salary: 91000
Id: 4, Name: Emily White, Designation: QA Engineer, Salary: 56000
Id: 5, Name: Michael Green, Designation: DevOps Engineer, Salary: 66000
Id: 6, Name: Rocky, Designation: Team Lead, Salary: 95000
```

List result set showing row after deletion.

```
Id: 1, Name: John Doe, Designation: Software Engineer, Salary: 61000
Id: 2, Name: Jane Smith, Designation: Senior Developer, Salary: 76000
Id: 3, Name: Robert Brown, Designation: Project Manager, Salary: 91000
Id: 4, Name: Emily White, Designation: QA Engineer, Salary: 56000
Id: 5, Name: Michael Green, Designation: DevOps Engineer, Salary: 66000
```